

AIR ANALYZER

Low-power Respiratory Monitoring Mask for Athletes

Team Members: Anna Dal Mas , Alessia Rinaldi



OBJECTIVE



To design and prototype a lightweight, low-powered wearable device for monitoring of respiratory activity during exercise.



The system will operate through button battery, low power microcontroller and pressur sensor to acheive low power operation.



Collected data will be analysed to compare respiratory responses to physical exertion between trained athletes and untrained individuals.



HOW DOES IT WORK?

01

Breath Detection

- The user breathes in/out wearing the mask.
- A pressure sensor detects subtle changes in air pressure inside the mask

02

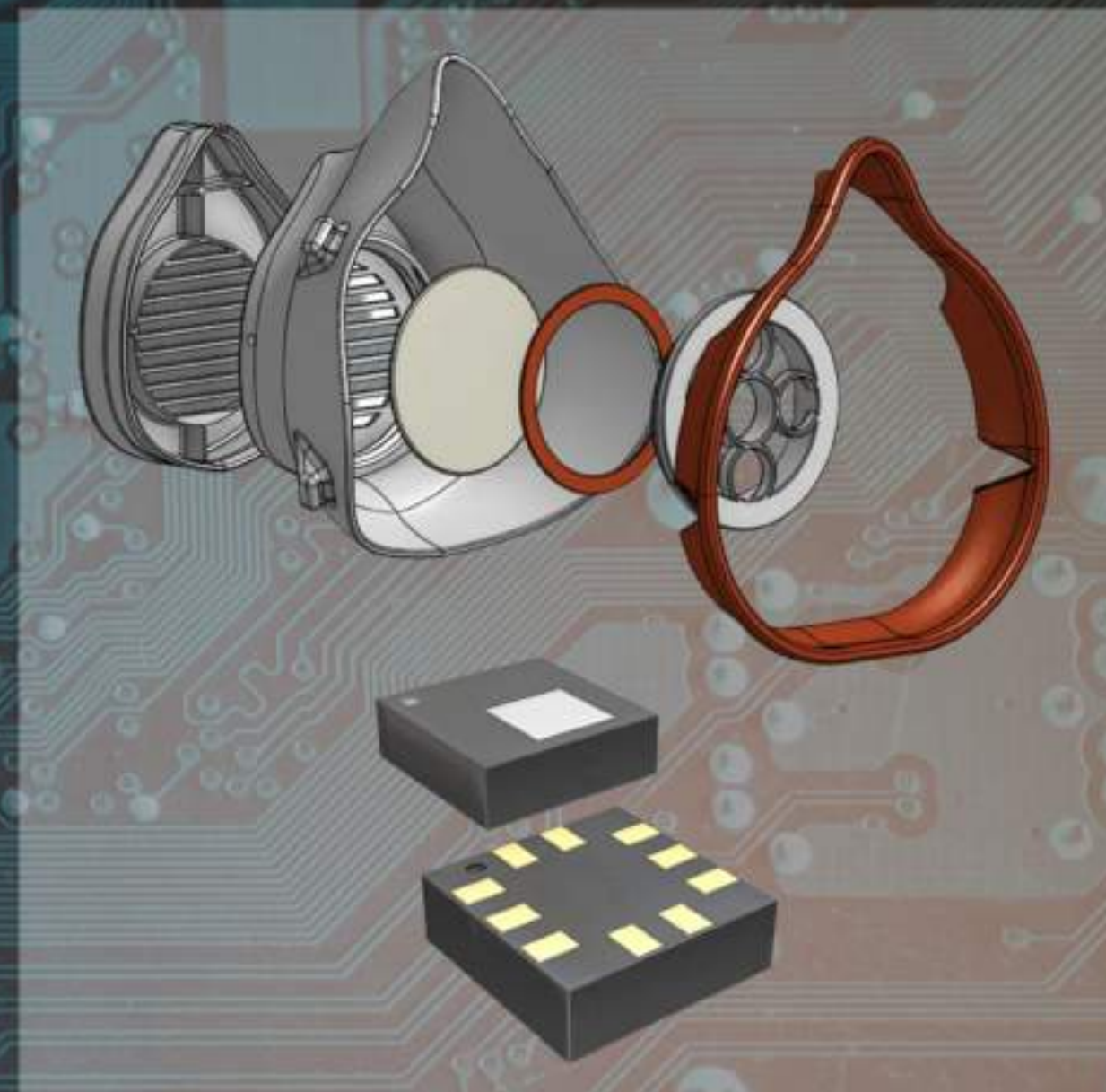
Data Processing

- The microcontroller (MCU) reads sensor data (via SPI).
- Breath patterns are processed (e.g., counting breath cycles, measuring strength)

03

Power Management

- The device is powered by a CR2032 coin cell battery
- A TPS61291 boost converter steps up the battery voltage to power the system
- The microcontroller enters STOP mode between measurements to minimize power consumption and maximize battery life



HOW DOES IT WORK?

04

Data Storage

- Pressure data is continuously saved to an external SPI EEPROM memory during the session
- The MCU wakes up only when the sensor FIFO buffer is full (every ~4 seconds at 8Hz, since the FIFO has 32 samples)

05

Data Download

- After the measurement session, data is downloaded to a PC via USB-C
- No real-time transmission — data is analyzed offline after the session

05

Visualization

- Downloaded data is logged and visualized for analysis (athlete vs. non-athlete)



FROM CONCEPT TO PROTOTYPE:

WHAT WE WILL BUILD

01

3D-Printed Mask

- Design and print a custom mask structure
- Ensure comfort and usability for real testing

02

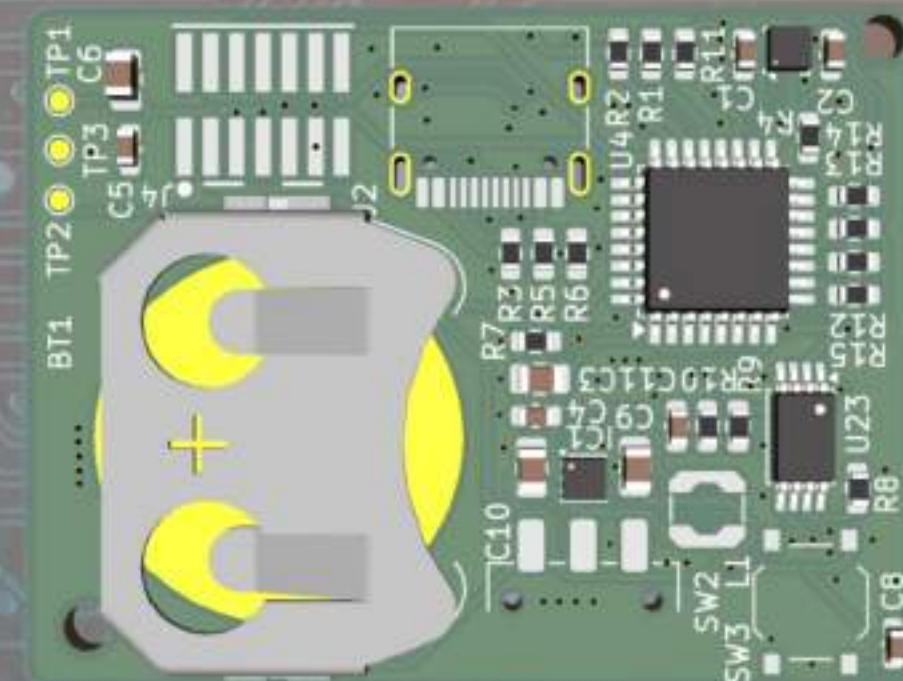
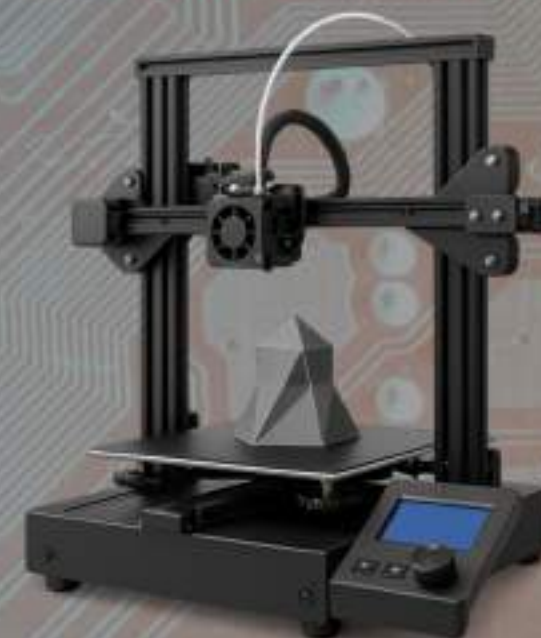
PCB Development

- Design and fabricate a low-power PCB
- Integrate microcontroller, pressure sensor, EEPROM memory and USB-C connector
- Optimize power management using STOP mode and sensor FIFO buffering
- Power supplied by CR2032 coin cell battery with boost converter

03

Testing and Data Analysis

- Test with two user profiles: 1. Trained/athletic individuals
2. Non-trained individuals
- Collect and compare breathing data

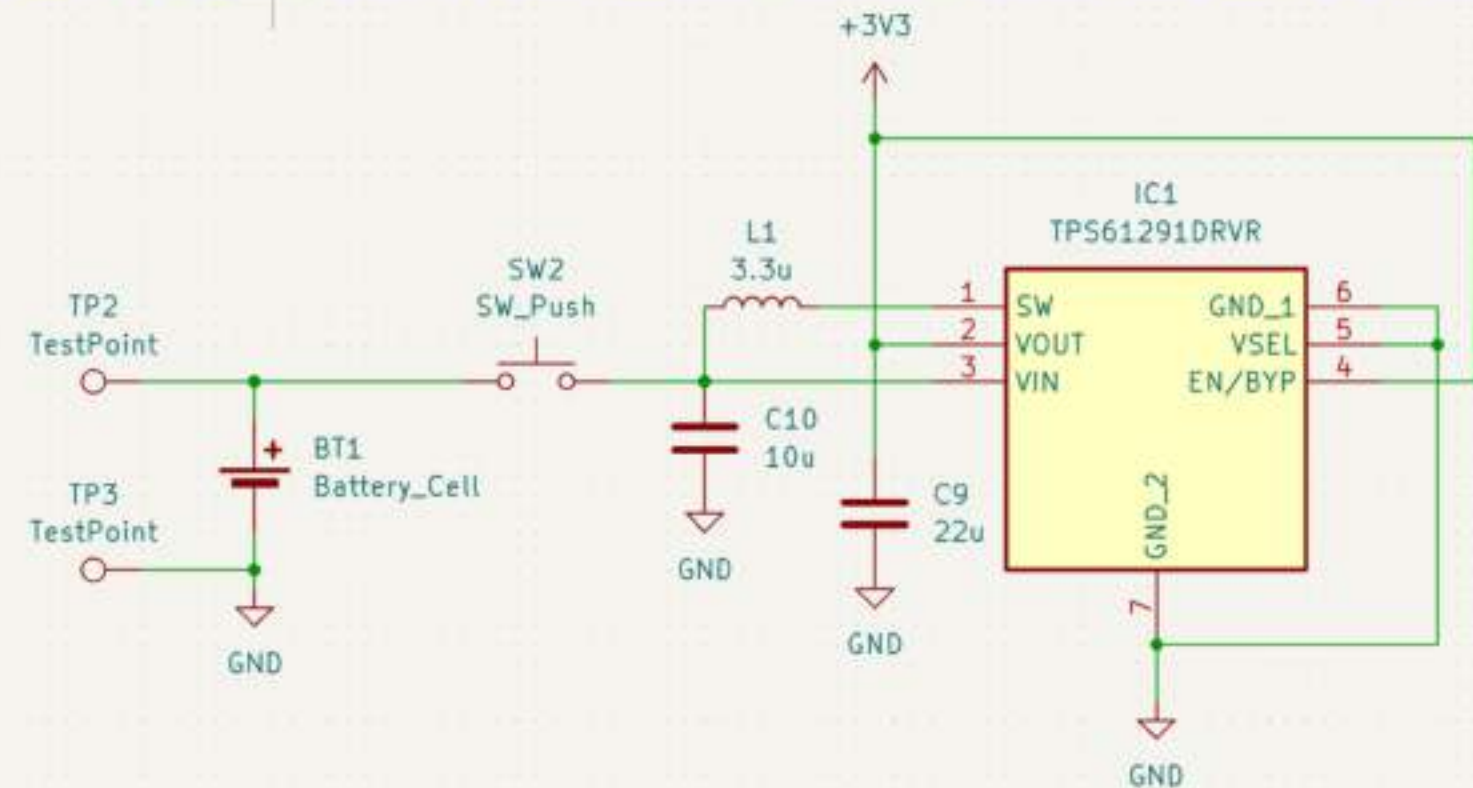


SCHEMATIC

Power Supply

- CR2032 coin cell battery as power source
- TPS61291 boost converter steps up battery voltage to regulated 3.3V
- SW2 slide switch for power on/off
- Input capacitor (10 μ F) and output capacitor (22 μ F) for voltage stability
- TP2, TP3 test points for debugging

Power supply 3V $\rightarrow$ 3.3V

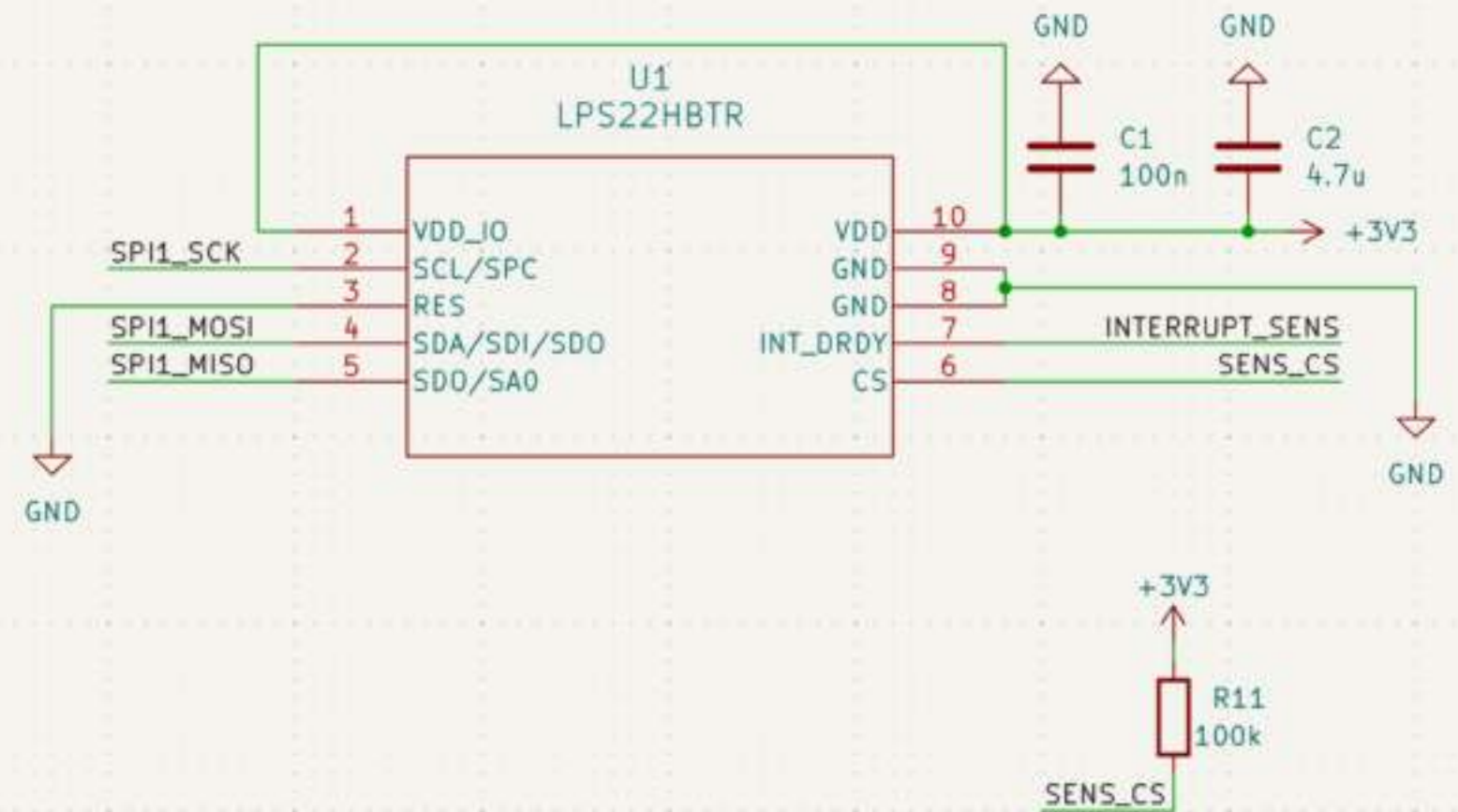


SCHEMATIC

Pressure sensor

- Barometric pressure sensor communicating via SPI
- INTERRUPT_SENS signal connected to STM32 for FIFO threshold notification
- SENS_CS chip select for SPI bus management
- Decoupling capacitors (100nF, 4.7μF) on VDD for noise filtering

Pressure Sensor



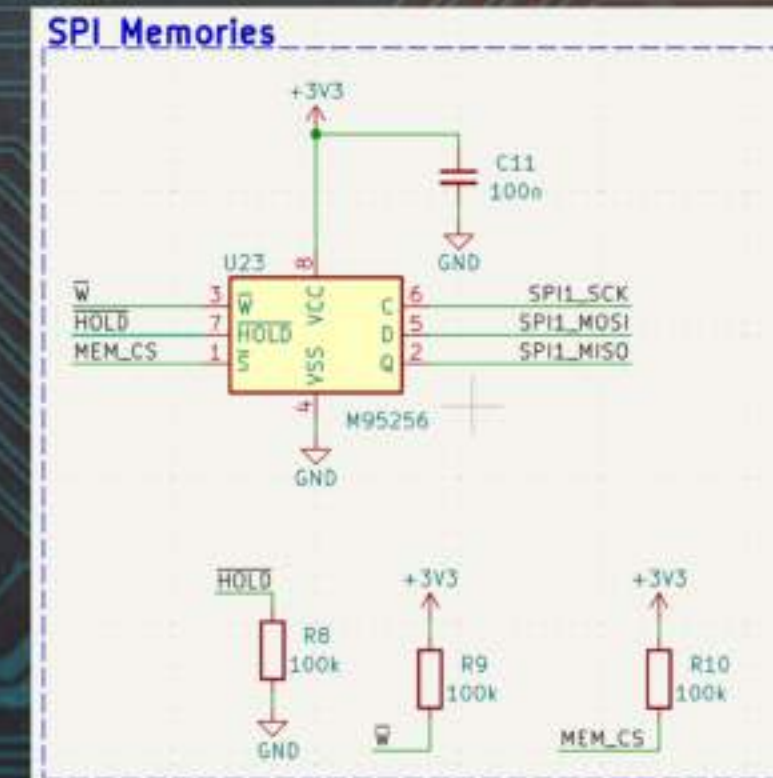
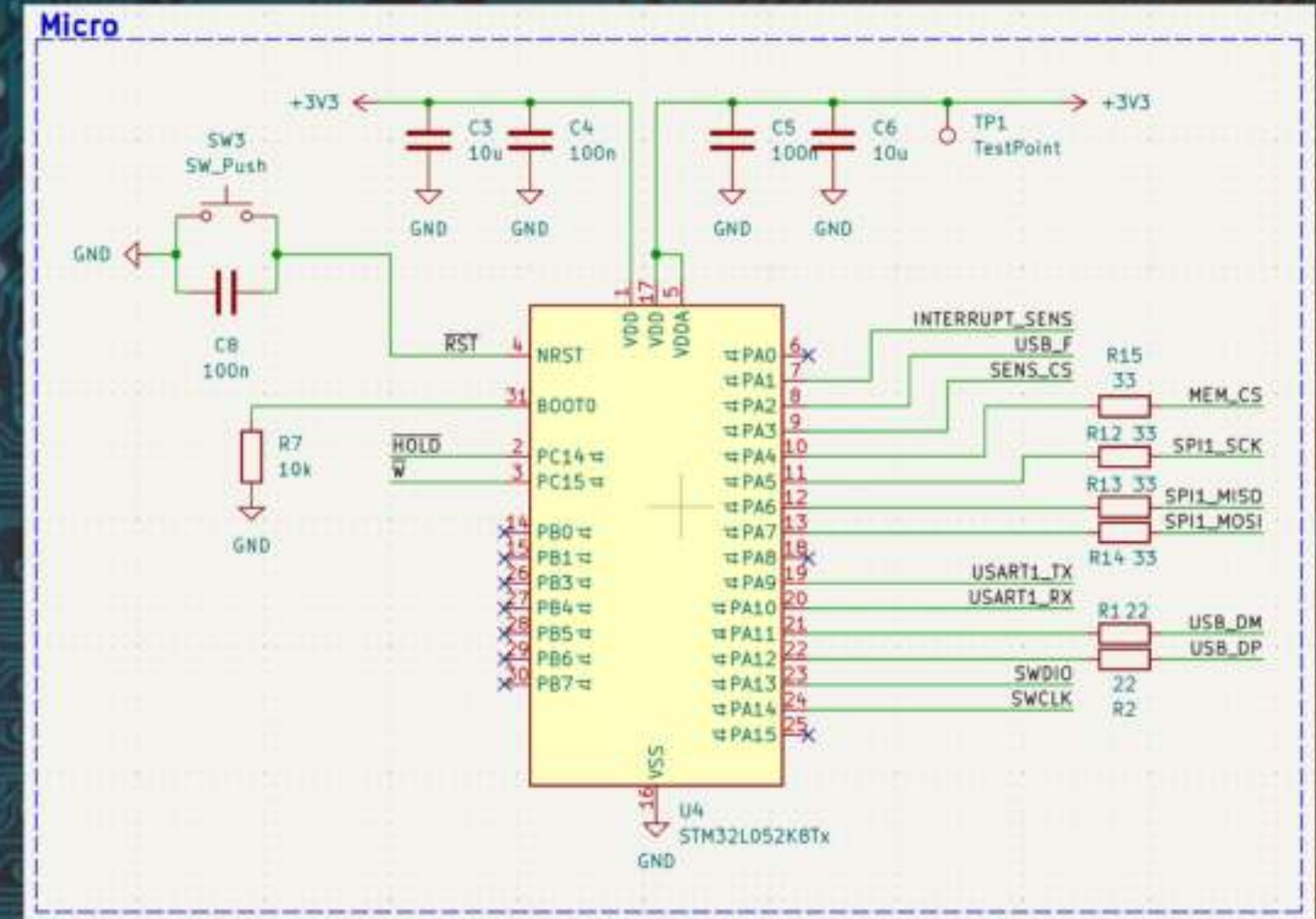
SCHEMATIC

Microcontroller

- Low-power ARM Cortex-M0+ microcontroller
- SW3 push button for reset
- Multiple decoupling capacitors on VDD/VDDA for stable power supply
- SPI interface for sensor and memory communication
- USART1 for serial communication
- SWD interface (SWDIO/SWCLK) for programming and debugging
- Series resistors (33Ω) on SPI lines for signal integrity
- R7 pull-down on BOOT0 for normal boot mode

SPI memory

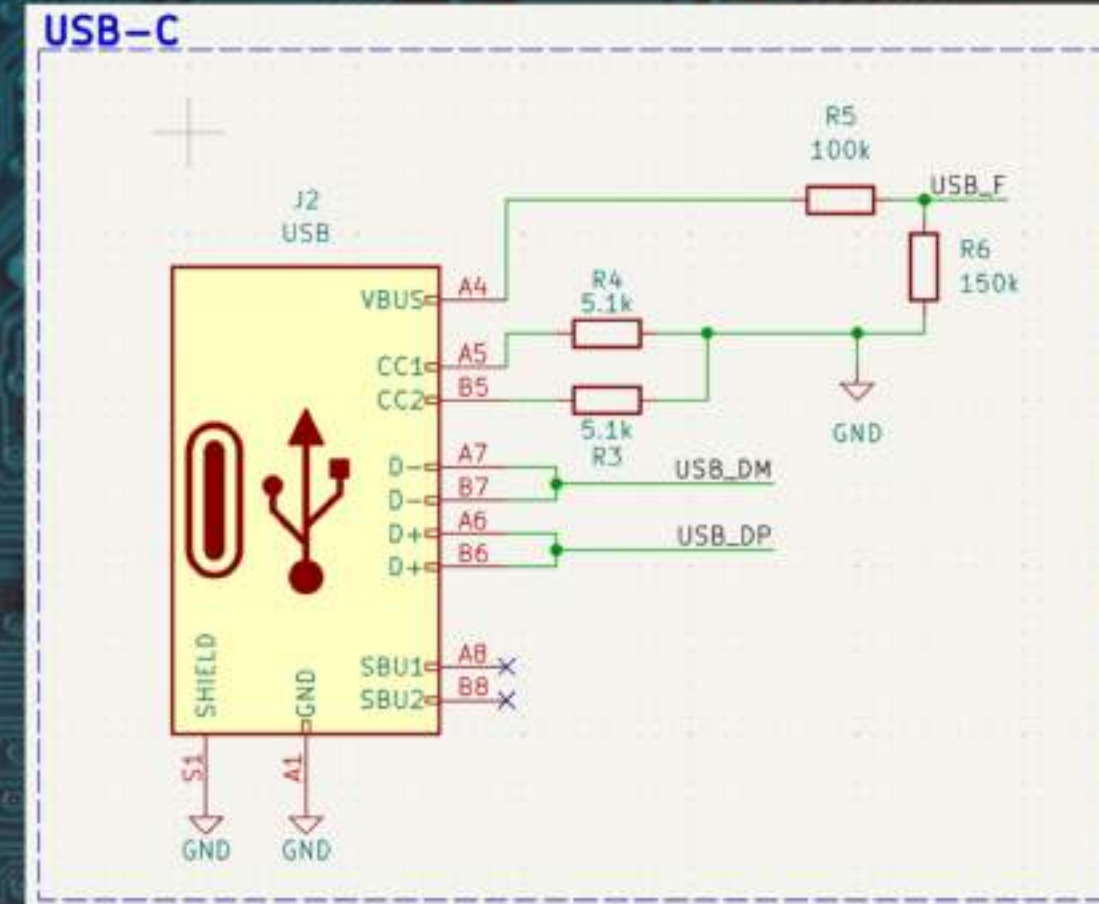
- 256Kbit SPI EEPROM for data storage (~18 minutes at 10Hz)
- HOLD pin managed via dedicated GPIO
- Write protection (W pin) controlled via resistor network
- MEM_CS chip select separate from sensor CS
- Decoupling capacitor (100nF) on VCC



SCHEMATIC

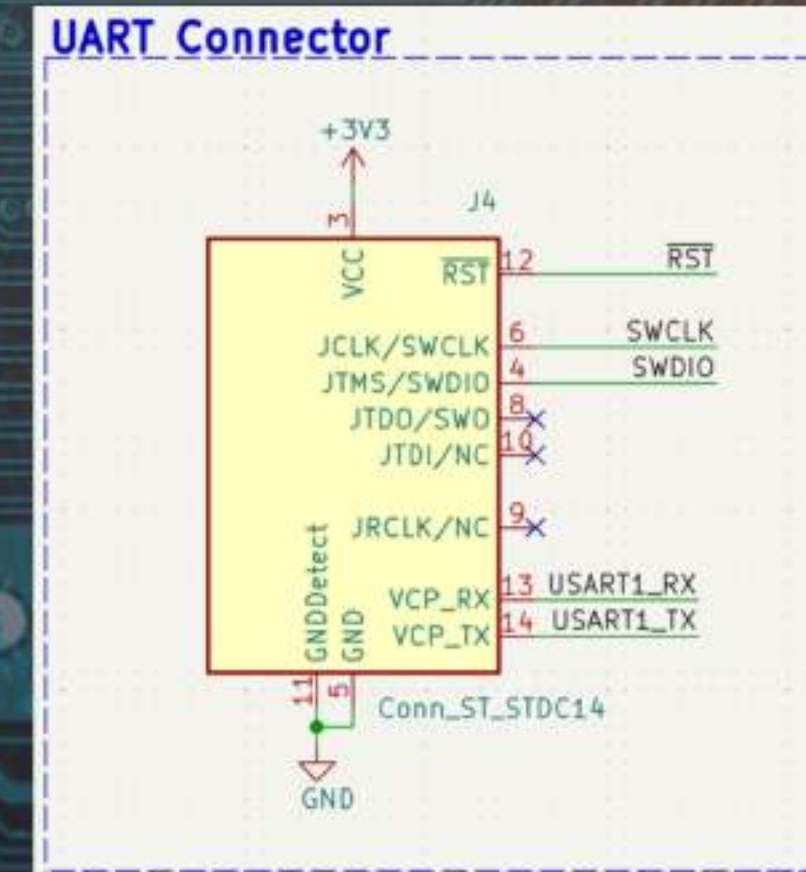
USB-C connector

- USB-C connector (J2) for data download to PC
- CC1/CC2 resistors (5.1k Ω) for USB-C configuration
- R5/R6 pull-up resistors on data lines
- ESD protection on shield pins

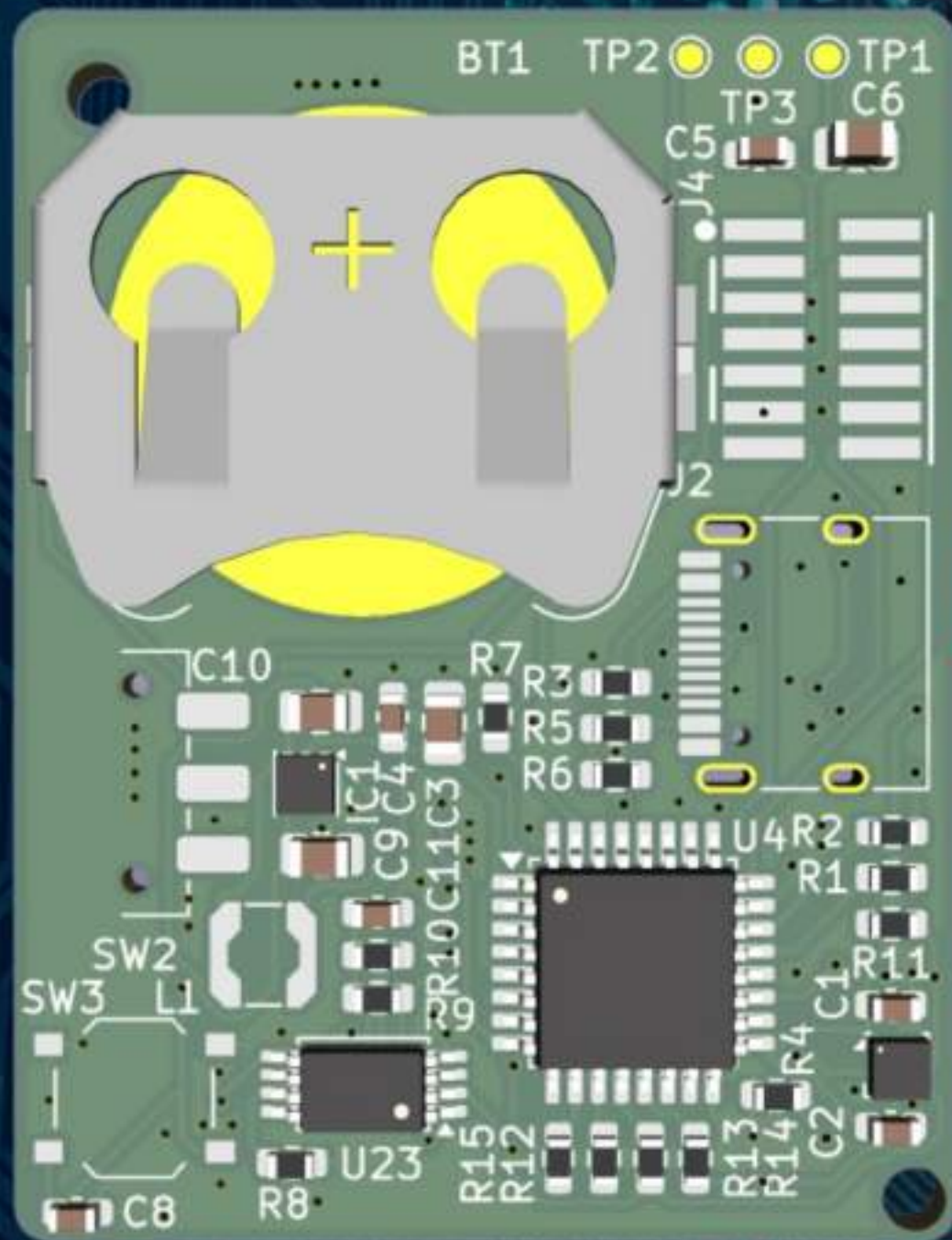


UART connector

- 14-pin ST standard debug connector
- Provides SWD interface for STM32 programming
- USART1 TX/RX for serial communication with PC
- Hardware reset line (RST) accessible from connector
- VCC pin for board powering during development



PCB



DATA ACQUISITION WORKFLOW

01 The sensor autonomously acquires data and fills its FIFO

02 When the FIFO reaches 25 samples, the sensor generates an interrupt

03 The microcontroller wakes up

04 The MCU uses SPI + DMA to read the 25 samples from the sensor

05 Once the DMA transfer is complete, an interrupt is triggered

06 The CPU stores the data into memory

07 The system then returns to sleep

SENSOR SAMPLING FREQUENCY SELECTION

- Resting respiratory rate: ~12–20 breaths per minute
- Respiratory rate under physical exertion: ~30–50 breaths per minute

- According to the Nyquist criterion: $f_s \geq 2f_{max}$ Sampling frequency: $f_{max} \approx 0.8 \text{ Hz}$
 $f_s \geq 1.6 \text{ Hz}$

SENSOR SAMPLING FREQUENCY SELECTION

FINAL DECISION

- We select a sampling frequency of 10 Hz.
Given a FIFO depth of 32 samples (each 40 bits), the FIFO is filled every:

$$\frac{32}{10} = 3,2 \text{ seconds}$$

- This allows the microcontroller to wake up less frequently, reducing overall power consumption.

MEMORY USAGE AND MAXIMUM OPERATING TIME

- The system uses a 32 kB external EEPROM, while the PCB is designed to support also a 64 kB. The 32 kB version was selected for cost reduction during production.
- Based on the previously defined sampling parameters, the maximum operating time of the device can be estimated.
- Each measurement consists of:
 - 3 bytes for pressure data
 - 2 bytes for temperature data (not used)
- If all 3 bytes of pressure data are stored:
 - 32 kB memory allows approximately 18 minutes of continuous operation
 - 64 kB memory would allow approximately 36 minutes

FIRMWARE ARCHITECTURE

Environment developed using STM32CubeMX for peripheral configuration (GPIO, SPI, DMA, USART, USB, clk)

01

Initialization (runs once at startup)

- Configure peripherals and communication interfaces
- Set LPS22HB for continuous acquisition at 10 Hz
- Start first SPI/DMA transfer → pipeline becomes interrupt-driven

02

Low-Power Main Loop

- Main loop executes `__WFI()` (Wait For Interrupt)
- CPU sleeps while DMA handles SPI transfers autonomously
- Processor wakes only when DMA transfer-complete interrupt fires
- Result: CPU active only for brief data-handling intervals

03

Interrupt Handler (runs after each DMA transfer)

- Extracts raw pressure bytes from reception buffer (3 bytes / sample)
- No on-board conversion to physical values → minimal computational overhead
- Writes raw 24-bit sample directly to M95256 EEPROM
- Starts next sensor acquisition → cyclic pipeline restarts